

LAB 4

Sincronizzazione in OS161

Programmazione di Sistema (IA-ZZ)

2024/25

Giorgio Cora

Obiettivi:

- Introduzione alla sincronizzazione in os161
- Implementazione dei lock
- Implementazione delle condition variables

In os161 sono presenti 3 principali direttive di sincronizzazione:

- Spinlock
- Semafori
- Waitchannels

Semafori e spinlock permettono la sincronizzazione di thread e processi. Uno spinlock è essenzialmente un semaforo binario.

Gli spinlock si basano sulle funzioni `spinlock_acquire(splk)` e `spinlock_release(splk)`. I semafori si basano sulle funzioni `P(s)`, che esegue la wait e si usa nel momento in cui si entra in una sezione critica, e `V(s)`, che segnala l'uscita dalla sezione critica.

Quello che manca sono i lock e le condition variables.

I lock NON sono implementati in os161. Tuttavia è disponibile la loro interfaccia:

```
struct lock *myLock = lock_create("LockName");  
lock_acquire(myLock);  
    critical section  
lock_release(myLock);
```

Il lock introduce il concetto di ownership: il thread che acquisisce il lock DEVE essere lo stesso che lo rilascia.

Un po come gli spinlock, anche Ilock sono una sorta di semaforo binario. Quello che li differenzia è ciò che succede ad un thread nel momento in cui non riesce ad acquisire un lock (viene messo in sleep) oppure uno spinlock (esegue busy waiting).

Per implementare i lock si possono seguire due strade:

1. Utilizzando i semafori. È possibile modificare la struttura lock in **kern/include/synch.h** e completando le funzioni **lock_create()**, **lock_destroy()**, **lock_acquire()**, **lock_release()**, **lock_do_i_hold()** in **kern/thread/synch.c**.
2. Utilizzando le funzioni spinlock e wchan e adattandoli alla realizzazione interna dei semafori (creare una nuova versione di semafori binari).

Una condition variable è una primitiva di sincronizzazione che consente di attendere che una condizione (eventualmente falsa al momento attuale) diventi vera. È sempre accompagnata da un lock, passato come parametro alle funzioni, che ne garantisce l'accesso protetto in mutua esclusione.

- **cv_wait():** rilascia il lock (che deve essere acquisito in precedenza dal thread chiamante) e si mette in attesa di cv_signal() o cv_broadcast().
- **cv_signal** e **cv_broadcast():** svolgono lo stesso compito ma differiscono nel numero di thread svegliati (uno e tutti, rispettivamente).

Si richiede la realizzazione della condition variable mediante **wait_channel** e **spinlock**.

IMPORTANTE: `wait_channel` ha una semantica simile a quella della condition variable ma ha uno `spinlock` invece del `lock`.

I programmi di test che utilizzano le condition variables sono **sy3** e **sy4**, che chiamano le funzioni **cvtest()** e **cvtest2()**.

Le definizioni delle funzioni sulle condition variables sono in *kern/include/synch.h* e *kern/thread/synch.c*.